Part 4 Onboarding Guide

Back Office | POS Project (video game store) | Prepared for Mo

Prepared Oct 20, 2026 (Day 5 of the plan). Based on the Part Briefs (Oct 16), the current schema design document (updated Oct 16, evening) and the project plan. Anything the lead decided after Oct 16 is not in here, so ask the lead if something looks different.

1. At a glance

Item	Detail
Your part	Part 4: Back office. You give employees the tools to run the store: inventory, prices, discounts, low-stock alerts and supplier orders. Managers are the only role allowed to change inventory, prices and discounts.
Branch	`part-4-backoffice`. Pull requests go to the lead, who merges to main. Vercel deploys from main only.
Folders you own	`db/backoffice/`, `api/manager/`, `api/reports/inventory.js`, `src/pages/backoffice/`, and your own inventory report file in `src/pages/reports/`.
You build	1 trigger (`TR_INVENTORY_ADJUSTMENT_Apply`) with its SQL tests; the `api/manager/` areas (products, prices, adjustments, discounts, sale-events, suppliers, supplier-orders, alerts); the inventory report endpoint; 6 screens plus the sale event screens; and a Jest test per endpoint.
Deadlines already passed (today is Oct 20)	Foundation ready (Oct 16 to 17), **schema freeze (Oct 18, end of Day 3)** and the walking skeleton (Oct 19, end of Day 4). Because the schema is frozen, you cannot change tables without the lead's approval.
Deadlines still ahead	Your `triggers.sql` and `test.sql` to the lead by **Day 6 (Oct 21, tomorrow)**. Full chain running through the app on Day 7 (Oct 22). Feature freeze Oct 23. Bug bash Oct 23 to 24. Submission Oct 25.
Never cut	The trigger, stock adjustment and price edit, the low-stock to supplier order to restock flow, the inventory report (only the inventory value part may be cut), manager-only enforcement. If you fall behind, tell the lead early: the lead takes over Part 4's alert-to-restock flow second, right after checkout.

2. Documents to read first

Document	How to use it
schema-current-design.md	**Source of truth** for tables, columns, triggers, decisions and the rebuild procedure. Newer than the conceptual schema PDF. It also contains the full text of the 5 built triggers.
POS Project: Part Briefs (PDF)	Your Part 4 page (page 7) plus the shared rules, the lead's deliverables, the timeline and the cut order.
Conceptual schema PDF	**Outdated. Ignore it for design.** It still has Register, Cashier, nullable Customer_id on transactions and cash payments, which were all removed.
Technology resources PDF	Tutorials for SQL Server, Node.js and React. Useful for learning, not for decisions.
triggers.sql, test_triggers.sql, schema file, drop.sql	SQL files kept by the lead. They are **not attached to this guide**; ask for them (Section 4). `triggers.sql` is the style model for your trigger and `test_triggers.sql` is the model for your `db/backoffice/test.sql`.

3. Install the tools

These steps assume Windows (SQL Server Express runs on Windows). Run commands in PowerShell or the VS Code terminal (Ctrl+`). You already have VS Code, a SQL Server Express connection from VS Code and access to the team repository, so those are skipped.

Step 1: Git

1. Download Git for Windows from https://git-scm.com/download/win and run the installer with the default options.
2. Close and reopen VS Code, then run `git --version`.
3. Set your identity (use your GitHub email):

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

Step 2: Node.js (LTS)

1. Download the **LTS** version from https://nodejs.org (not Current) and install with the defaults.
2. Reopen VS Code and check `node -v` and `npm -v`. Both must print a version.
3. If PowerShell says scripts are disabled when you run npm, run this once:

```
Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned
```

Step 3: Get the project and install its packages

> **Important.** Do **not** run `npm create vite` or start a new project. The lead owns the skeleton (Vite, React Router, lib/, src/shared/). You work inside the shared repo.

1. The team repository already exists and you have access. Clone it (use the repository URL from GitHub, under the green Code button) and move into the folder.
2. Create your branch.
3. Install everything listed in the lead's `package.json`.
4. Check the stack is present.

```
cd C:\Projects
git clone <repo-url>
cd <repo-folder>
git checkout -b part-4-backoffice
npm install
npm ls mssql bcrypt jsonwebtoken jest react-router-dom vite
```

- If a package is missing, **tell the lead** instead of committing a change to `package.json`, so the five parts do not conflict. To keep testing locally in the meantime: `npm install mssql bcrypt jsonwebtoken react-router-dom` and `npm install --save-dev jest vite`.
- If `bcrypt` fails to install (it is a native module), the lead can switch to `bcryptjs`. Otherwise install "Desktop development with C++" from the Visual Studio Build Tools and retry.

Step 4: Build your local pos_store database

Your trigger and its tests run against your own local database, so you need one that matches the schema. If you already built `pos_store` locally and the counts in Step 8 match, skip this step. Ask the lead for the schema file, `triggers.sql` and the seed script (they are not attached here).

1. In VS Code, connected to `master`, create the database. Run this once.

```
CREATE DATABASE pos_store;
```

2. Run the **schema file**, then `triggers.sql`, then the **seed script**, **each exactly once**. Both SQL files start with `USE pos_store;`. Running the schema file twice gives "There is already an object named ..." for every table.
3. The seed script ends with `UPDATE PRODUCT SET Stock_quantity = Stock_quantity;`. That line makes products that are already at or below their threshold get a low-stock alert, so you have alerts to work with.
4. To start over, drop the database from `master` and run the files again in the same order. First check that no other query tab is connected, because the first line disconnects them:

```
ALTER DATABASE pos_store SET SINGLE_USER WITH ROLLBACK IMMEDIATE;
DROP DATABASE pos_store;
CREATE DATABASE pos_store;
```

Step 5: Let Node connect to your local SQL Express

VS Code's SQL extension can connect in ways Node's `mssql` package cannot, so this step is often the real blocker.

1. Open **SQL Server Configuration Manager**. Under SQL Server Network Configuration, Protocols for SQLEXPRESS, right-click **TCP/IP** and choose Enable.
2. Under SQL Server Services, set **SQL Server Browser** to Automatic and Running (needed to find the named instance).
3. Restart **SQL Server (SQLEXPRESS)** in the same list.
4. Node needs a SQL login, not Windows authentication. In VS Code, on `master`, check the mode. A result of 0 means mixed mode is already on. If it returns 1, enable "SQL Server and Windows Authentication mode" in the server Properties, Security page, then restart the service.

```
SELECT SERVERPROPERTY('IsIntegratedSecurityOnly');
```

5. Create a local development login. This is for your machine only; choose your own password.

```
CREATE LOGIN pos_dev WITH PASSWORD = 'ChooseAStrongPassword!1';
GO
USE pos_store;
GO
CREATE USER pos_dev FOR LOGIN pos_dev;
ALTER ROLE db_owner ADD MEMBER pos_dev;
GO
```

6 Put your settings in `.env.local` in the repo root. Ask the lead for the exact variable names that `lib/db.js` reads, and make sure `.env*` is in `.gitignore`. **Never commit this file.**

Setting	Local value
Server	localhost
Instance	SQLEXPRESS
Database	pos_store
User / password	The pos_dev login you created
Encryption	encrypt false and trust server certificate true (local only; the Azure VM uses encrypt: true)

Step 6: Vercel CLI (optional)

It runs the `api/` functions locally the way they run in production. The functions are plain Node handlers, not Express.

1 Install and log in: `npm install -g vercel`, then `vercel login`.

2 The lead must first add you to the Vercel project. Then run `vercel link` inside the repo.

3 Do **not** pull production environment variables unless the lead says so, because that downloads real credentials. Use your own `.env.local`.

4 Run `vercel dev` for app and API together. If the lead's setup uses `npm run dev` for the front end only, use that for screen work and rely on Jest for the API.

Step 7: Optional VS Code extensions

- **ESLint** and **Prettier**, if the repo uses them.
- **Thunder Client** or **REST Client**, to call `api/manager` endpoints without a screen.

Step 8: Verify the setup

[ ]	`git --version`, `node -v` and `npm -v` each print a version.
[ ]	`git branch` shows `* part-4-backoffice`.
[ ]	`npm ls` shows no missing packages.
[ ]	Nested triggers are on: `SELECT value_in_use FROM sys.configurations WHERE name = 'nested triggers';` returns **1**. The triggers chain, so this is required.
[ ]	`SELECT COUNT(*) FROM sys.tables;` returns 27 and `SELECT COUNT(*) FROM sys.foreign_keys;` returns 43.
[ ]	`SELECT name, OBJECT_NAME(parent_id) AS on_table, is_disabled FROM sys.triggers ORDER BY name;` lists at least the 5 built triggers with `is_disabled = 0` (more if you built from a newer `triggers.sql`). If it already lists `TR_INVENTORY_ADJUSTMENT_Apply`, tell the lead (see Section 10).
[ ]	A small Node script (or the lead's `query()` from `lib/db.js`, if it exists) connects as `pos_dev` and runs `SELECT 1`.
[ ]	`npx jest` runs without crashing ("no tests found" is fine).
[ ]	`npm run dev` starts Vite and the page opens in the browser.

Troubleshooting. If the connection test fails, the usual causes are TCP/IP disabled, SQL Server Browser stopped, mixed mode off, or the service not restarted. Send the exact error message when asking for help.

4. What you need from the lead

[ ]	Vercel project access, if you want to use the CLI (you already have access to the repository).
[ ]	`lib/db.js` (`query(sql, params)` and `withTransaction(fn)`), `lib/auth.js` (`requireAuth(['manager'])(handler)`) and `lib/http.js` (`sendJson`, `sendError`). Names were proposed and became final at the end of Day 2 (Oct 17), so check what is really in the repo.
[ ]	`src/shared/` components and the API client (`api.get`, `api.post`), the **employee shell** and the **role checks**, plus employee login, so you can test as a manager and as support.
[ ]	The **seed script**: products with stock (some at or below their threshold, so alerts exist), employees of both roles, suppliers, completed sales, a TAX_RATE for every category in every test state, and exactly one reward program with tiers. It must end with `UPDATE PRODUCT SET Stock_quantity = Stock_quantity;`.
[ ]	The exact variable names for `.env.local`, and a least-privilege database login for the Azure VM (not sa) when you need it.
[ ]	The schema file, `triggers.sql`, `test_triggers.sql` and `drop.sql`. The schema file and the 5 built triggers are what Step 4 needs.
[ ]	The lead's answers to the questions in Section 10, especially whether `TR_INVENTORY_ADJUSTMENT_Apply` is already in the database.

5. Rules you must follow

Database rules (shared by all parts)

1. Triggers are AFTER, set-based and named `TR_<Table>_<Purpose>`. They start with `SET NOCOUNT ON; SET XACT_ABORT ON;` and fail with `THROW`. Every SQL file begins with `USE pos_store;` then `GO`.
2. Never edit `CUSTOMER.Loyalty_points` or `Tier_id` directly. Points come from a LOYALTY_TRANSACTION row, and the built trigger updates the rest. (Part 4 normally never touches these.)
3. Never recompute stock, points, tier or low-stock alerts in the app. The built triggers keep them in sync.
4. Every query is parameterized. Prices, tax and totals come from the database, never from the request.
5. A write that touches several tables runs in one database transaction, using the lead's helper.
6. Your trigger's tests go in `db/backoffice/test.sql`. Every failing case runs in its own transaction and rolls back.

Decisions already closed (do not reopen)

Topic	Decision
Who may write	**Manager** role is required for inventory, price and discount writes; support can view. The employee id always comes from the login token, never from the request body. Enforced in the app, not the database. The database stores the roles as 'Manager' and 'Support'; check how `lib/auth.js` spells them in the token.
Stock changes	Once `TR_INVENTORY_ADJUSTMENT_Apply` exists, the app inserts only the INVENTORY_ADJUSTMENT row and **never updates Stock_quantity directly**. `CHECK (Stock_quantity >= 0)` blocks negative stock.
Price changes	No price audit trigger. The app inserts the PRICE_CHANGE row in the same transaction as the `Sell_price` edit, because a trigger cannot know the employee.
Deactivate, do not delete	PRODUCT and DISCOUNT have `Is_active` (default 1). Products with sales history are deactivated, not deleted; the catalog shows only active products. A discount applies only when `Is_active = 1` and today is within its Start_date and End_date. A hard delete of a product with history is blocked by foreign keys, so show a readable message.
Discounts at checkout	Per line, checkout takes the **single largest saving** among the product discount and the category discount. No stacking. `DISCOUNT.Code` is unused (no promo code box), so do not add a code field to your discount screens.
Low-stock alerts	The database opens and closes them (`TR_PRODUCT_LowStock`). The app never inserts or resolves an alert. The one thing the app does is set `Order_id` when a supplier order is created.
Supplier orders	Nothing is ordered automatically. Status goes Pending, Ordered, Received; Cancelled is allowed from Pending or Ordered; items can change only while Pending. **The app enforces this; the database does not.**
Schema	Frozen since Oct 18. Ask the lead before any table change. Errors 50001 and 50002 are already used by built triggers; pick a different `THROW` number and tell the lead.

> **Watch out.** Two ways this part can silently break another part. (1) If the app also updates `Stock_quantity` after the adjustment trigger exists, every adjustment is applied twice. (2) `TR_SUPPLIER_ORDER_Received` adds stock whenever a status changes to Received from *any* other status, including Cancelled or Pending, so an invalid status change from your API adds stock that never arrived.

Downstream, Part 1's catalog and Part 2's `sp_Checkout` read your `Sell_price`, `Is_active` and discounts, and Part 3's refund restock goes through the same low-stock trigger. A wrong price or an active discount that should be paused shows up in their screens.

6. The data you work with

Figure 1, on the last page of this guide, draws these tables and how they connect. Read the table below for the details of each column, then use the figure to see the relationships.

Table	What Part 4 needs to know
PRODUCT	Product_id, SKU (unique), Name, Description, Cost_price, Sell_price, Stock_quantity, Reorder_threshold, Product_condition ('New', 'Used', 'Refurbished'), Category_id, Discount_id (nullable), Is_active. `Stock_quantity` changes only through triggers. A product edit must never put `Stock_quantity` in its UPDATE. Changing `Reorder_threshold` fires `TR_PRODUCT_LowStock`, which may open or close an alert; that is expected.

Table	What Part 4 needs to know
PRICE_CHANGE	Price_change_id, Product_id, Employee_id, Old_price, New_price, Changed_at. Inserted by the app in the same transaction as the `Sell_price` edit. Read the old price inside that transaction.
INVENTORY_ADJUSTMENT	Adjustment_id, Product_id, Employee_id, Quantity_change (cannot be 0), Reason, Adjusted_at. Insert only; your trigger applies the change to stock. No other trigger touches it, and the app never edits or deletes these rows.
LOW_STOCK_ALERT	Alert_id, Product_id, Triggered_at, Resolved_at, Order_id (nullable FK). Open alert = `Resolved_at IS NULL`; its age is now minus `Triggered_at`. The triggers leave `Order_id` NULL, so the app sets it.
SUPPLIER	Supplier_name, Supplier_email (unique), Supplier_phone (exactly 10 digits, no dashes), Street_address, Apt_number (the only nullable column), City, State (2 letters, e.g. 'TX'), Zip_code (digits only, 5 to 10 characters, no hyphen). Every INSERT must supply all NOT NULL columns.
SUPPLIER_PRODUCT	PK (Supplier_id, Product_id), Supplier_sku, Unit_cost, Lead_time_days. Which supplier can supply which product.
SUPPLIER_ORDER	Order_id, Supplier_id, Employee_id (from the token), Order_date, Received_date (filled by the trigger when Received), Status ('Pending', 'Ordered', 'Received', 'Cancelled').
SUPPLIER_ORDER_ITEM	PK (Order_id, Product_id), Quantity, Unit_cost. One row per product in an order. Receiving the order adds these quantities to stock through the trigger.
DISCOUNT	Discount_name, Discount_type, Code (unique when not null; unused), Amount, Amount_type ('percent' or 'fixed'), Start_date, End_date, Sale_event_id (nullable), Is_active. Never hard delete one: deleting it sets `PRODUCT.Discount_id` and `CATEGORY.Discount_id` to NULL, which silently strips it from products and categories.
SALE_EVENT	Sale_event_id, Event_name, Description, Start_date, End_date, Is_active. Groups discounts. Deleting one sets `DISCOUNT.Sale_event_id` to NULL, so prefer Is_active.
EMPLOYEE, CATEGORY	Read only for you. EMPLOYEE gives names and the Role ('Manager', 'Support'). CATEGORY gives names for the product screens.
PRODUCT_RETURN (not drawn)	Read only, owned by Part 3. The dashboard counts rows with Status 'requested' for "returns waiting".

What already happens with stock (do not rebuild any of it)

1. A completed sale subtracts stock (`TR_SALES_TRANSACTION_Complete`). That stock change fires **TR_PRODUCT_LowStock**, which opens a LOW_STOCK_ALERT when stock is at or below the threshold and the product has no unresolved alert (no duplicates).
2. A manager creates a supplier order **by hand** (your code) and moves it Pending, Ordered, Received.
3. Changing the status to Received fires **TR_SUPPLIER_ORDER_Received**: it adds each item's quantity to stock and fills Received_date if empty. Re-saving an already received order does not add stock twice.
4. That stock change fires **TR_PRODUCT_LowStock** again, which sets Resolved_at once stock is above the threshold. Receiving too little leaves the alert open.
5. Your new trigger does the same job for manual adjustments: the insert changes stock, and `TR_PRODUCT_LowStock` opens or closes alerts. This is why nested triggers must be on.

7. Build order

Do the SQL first, then the API, then the screens. The trigger is small, and the lead needs it by tomorrow. After that, build the never-cut alert-to-restock flow before the discount work.

Phase 0: Sync with the lead (before writing code)

- Get the files and answers from Sections 4 and 10, and confirm the helper names in `lib/`.
- Confirm that `TR_INVENTORY_ADJUSTMENT_Apply` is not already in the database (the schema document contradicts itself on this).
- Tell the lead the real ETA for the SQL (due Oct 21) if you are behind.

Phase 1: The trigger and its tests (db/backoffice/)

Trigger	Table and event	Rule
TR_INVENTORY_ADJUSTMENT_Apply	INVENTORY_ADJUSTMENT, AFTER INSERT	Apply Quantity_change to the product's Stock_quantity and **reject a negative result**. Every manual stock change then leaves an audit row that already carries the Employee_id.

- Set-based: a multi-row insert, even two rows for the same product, must sum the changes per product through `inserted` and apply them once. Use `CREATE OR ALTER TRIGGER` like `triggers.sql` does, and put it in `db/backoffice/triggers.sql` starting with `USE pos_store;` then `GO`.
- Suggestion: check the resulting stock first and `THROW` a clear message, instead of letting the stock CHECK fail with error 547. Keep the CHECK as the backstop.
- Do not edit the 5 built triggers. The stock update you make will fire `TR_PRODUCT_LowStock` by itself.

Tests go in `db/backoffice/test.sql`. Use `test_triggers.sql` as the model: it creates its own employee, category and product (stock 10, threshold 3), then rolls back. Each failing case runs in **its own transaction** that is rolled back, because a failing statement leaves the transaction unusable.

Test	Expected result
Positive adjustment (for example +5)	Stock goes up by exactly 5
Negative adjustment that stays at 0 or above	Stock goes down by exactly that amount
Negative adjustment that would go below 0	Rejected; stock unchanged (own transaction)
Quantity_change of 0	Rejected by the table's CHECK (own transaction)
Two adjustment rows for one product in one INSERT	Both are applied, once each

Recommended extras: an adjustment down to the threshold opens one alert, and an adjustment back above it closes the alert; and the end-to-end flow from the briefs (a sale drops stock to the threshold, an alert opens, a supplier order is created and received, stock rises and the alert closes). A pattern for a failing case (suggestion; copy the real style from `test_triggers.sql`):

```
BEGIN TRANSACTION;
BEGIN TRY
    -- insert the test product, then an adjustment that makes stock negative
    INSERT INTO INVENTORY_ADJUSTMENT
        (Product_id, Employee_id, Quantity_change, Reason, Adjusted_at)
    VALUES (@productId, @employeeId, -999, N'test', SYSDATETIME());
    PRINT 'FAIL: negative stock was accepted';
END TRY
BEGIN CATCH
    PRINT 'PASS: rejected - ' + ERROR_MESSAGE();
END CATCH;
IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
```

When done, send `triggers.sql` and `test.sql` to the lead. Until the lead applies your trigger to the shared database, test against your local database, and **do not write a fallback that updates stock in the app**.

Phase 2: Products, prices and stock (api/manager/)

- **Products**: create, edit, activate and deactivate. Writes are manager only; support can read. The edit statement never lists `Stock_quantity`. Product delete is cut-first; if you build it, catch the foreign key error (also number 547, so check the message, because the stock CHECK uses it too) and show a readable message that suggests deactivating.
- **Prices**: in one transaction read the current `Sell_price`, update it, and insert the PRICE_CHANGE row with the token's Employee_id.
- **Adjustments**: insert the INVENTORY_ADJUSTMENT row only (Quantity_change not 0, a Reason, Employee_id from the token), then read the new stock back to show it.
- Screens: inventory list with adjust, and product edit with price. Each handles loading, empty and error states. A Jest test per endpoint, including one that refuses support on writes.

Decide and write down before coding

- How a new product gets its first stock: an INSERT with a stock value, or stock 0 and then an adjustment with the reason 'Initial stock' so the audit row exists. Note that a new product at or below its threshold opens an alert at once.
- Which Reason values the adjustment screen offers (the column is free text).
- Which validations the API adds: prices not negative, percent between 0 and 100, a fixed amount not above the price, end date not before start date.
- Whether the inventory list shows inactive products (a filter is a good default).

Phase 3: Alerts and supplier orders (the never-cut flow)

1. **Alerts list**: open alerts with product, stock, threshold, age and the linked order. Read only; the database opens and closes them.
2. **Create a supplier order** in one database transaction: insert SUPPLIER_ORDER as 'Pending' with the token's Employee_id, insert its items, then link the alerts for each ordered product.

```
UPDATE LOW_STOCK_ALERT SET Order_id = @orderId
WHERE Product_id = @productId AND Resolved_at IS NULL;
```

3. **Change the status** only along the valid path. Check the current status in the same transaction, for example `UPDATE ... WHERE Order_id = @id AND Status = @current` and refuse if no row changed.
4. **Edit items** only while the order is Pending. For Received, change only the status; the trigger adds the stock, so the app never touches `Stock_quantity`.

From	Allowed next status
Pending	Ordered, Cancelled
Ordered	Received, Cancelled
Received, Cancelled	None (final)

- Suppliers and supplier-product lists: the lists are must-have; the management screens (create and edit) are cut-first. Use the SUPPLIER insert shape from the schema document: `INSERT INTO SUPPLIER (Supplier_name, Supplier_email, Supplier_phone, Street_address, City, State, Zip_code) VALUES (...)`.
- Suggestion: validate Quantity above 0 and Unit_cost not negative in the API, and default an item's Unit_cost from SUPPLIER_PRODUCT when one exists.
- Screens: supplier orders (list, detail, status buttons that only offer valid next steps). Jest tests include an invalid transition and a second "Received" click.

Phase 4: Discounts and sale events

- **Discounts**: create, edit and pause with `Is_active` (never delete); set the percent or fixed amount and the dates; attach a discount to a product through the product edit (`PRODUCT.Discount_id`). No promo code field.
- **Sale events** (cut-first): create, edit and pause; group discounts with `DISCOUNT.Sale_event_id`. The sale event screens are also cut-first.
- Screens: discounts; sale events if time allows. Manager only for writes. Jest tests for both endpoints.

Phase 5: Inventory report

- `api/reports/inventory.js` and your own file in `src/pages/reports/`: stock against threshold per product, open alerts with their age, supplier orders by status, and inventory value at cost (cut-first; for example the sum of `Cost_price * Stock_quantity`). Add a Jest test, and confirm with the lead which roles may open it.

Phase 6: Dashboard and finish

- Employee dashboard: open alerts, pending supplier orders and returns waiting (a read-only count of PRODUCT_RETURN rows with Status 'requested').
- Run the whole flow against the real chain on your local database: a sale drops stock, an alert opens, you order, receive, and the alert closes. Then run your tests with the lead's chain and open a pull request from `part-4-backoffice` to the lead.

8. Schedule, pacing and cut order

Date	Plan milestone	Suggested pacing for Part 4
Oct 20 (Day 5, today)	Parts 1 to 4 build (Oct 17 to 22)	Section 3 install and local database; Phase 0 and Phase 1 (trigger and tests)
Oct 21 (Day 6)	**Your triggers.sql and test.sql due to the lead**	Send the SQL first thing; Phase 2 (products, prices, stock)
Oct 22 (Day 7)	Last build day; full chain through the app	Phase 3 (alerts and supplier orders) working end to end; start Phase 4
Oct 23 (Day 8)	**Feature freeze**	Finish Phase 4, then Phases 5 and 6; pull request
Oct 23 to 24	Test and bug bash	Fix bugs found by the team
Oct 25	Submission	One .bak or .sql file (schema, then triggers)

The pacing column is a suggestion built from the plan dates; the middle column comes from the project plan. About three and a half working days remain, so cut early rather than late. If the end-to-end flow is not working by the end of Oct 22, tell the lead right away.

If you fall behind, cut in this order

1. Sale event screens, then sale events and supplier management (the queries and screens).
2. Product delete and the inventory value in the report.
3. Then tell the lead: the lead takes over the alert-to-restock flow.

Never cut `TR_INVENTORY_ADJUSTMENT_Apply`, the low-stock to supplier order to restock flow, or the inventory report.

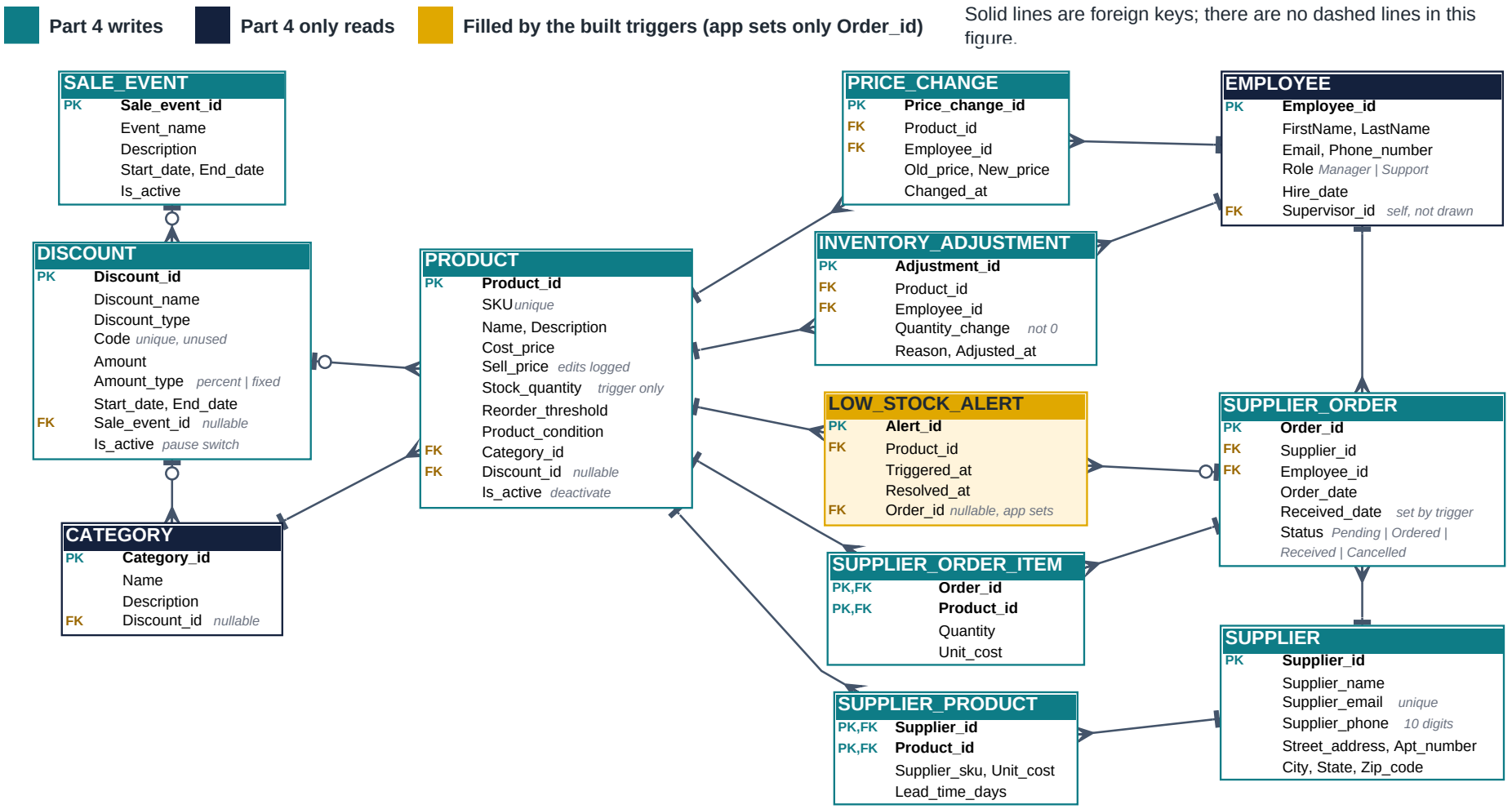
9. Definition of done

[ ]	`TR_INVENTORY_ADJUSTMENT_Apply` is built, named correctly, set-based, and passes its tests in `db/backoffice/test.sql` (failing cases each in their own rolled-back transaction).
[ ]	The app never updates `Stock_quantity` directly, and no product edit lists it.
[ ]	Each endpoint area (products, prices, adjustments, discounts, sale-events, suppliers, supplier-orders, alerts, inventory report) has a Jest test that calls the handler directly, including a refused support or non-manager write.
[ ]	Supplier order status changes outside the valid path are refused, and items change only while Pending.
[ ]	Creating a supplier order links the open alerts through `Order_id` in the same transaction.
[ ]	The end-to-end flow works: alert opens, order, Received, stock rises, alert closes.
[ ]	Each screen handles loading, empty and error states.
[ ]	Employee id comes from the token, writes are manager only, and nothing is hard deleted.
[ ]	No secrets are committed. The pull request is open against the lead.

10. Questions to confirm with the lead

- Is `TR_INVENTORY_ADJUSTMENT_Apply` already in the database? The schema document says in one table that the three C2 triggers are built, but its status line, section C and open items say only 5 triggers exist. This guide assumes it is not built.
- Which `THROW` numbers may Part 4 use (50001 and 50002 are taken), and where should `triggers.sql` and `test.sql` be sent?
- The exact variable names `lib/db.js` reads, and whether `lib/`, the employee shell and the seed script are merged yet.
- Roles: may support create or change suppliers and supplier orders, or manager only? May support open the inventory report? The briefs only say managers write inventory, prices and discounts and support can view.
- Should Part 4 assign discounts to categories? `CATEGORY.Discount_id` exists, but the endpoint list has no categories.
- What values does `DISCOUNT.Discount_type` take? The schema document gives no list.
- Does "pending supplier orders" on the dashboard mean Pending only, or Pending and Ordered?

Figure 1. The tables behind the back office and how they connect



Reading the lines. A bar means one parent row, a crow's foot means many child rows, and a circle means optional (NULL allowed). An alert can exist without an order; the app fills `Order_id` when it creates the order.

Two ways stock moves, both by trigger. Your INVENTORY_ADJUSTMENT insert and a supplier order set to Received (through SUPPLIER_ORDER_ITEM) change PRODUCT stock. That fires `TR_PRODUCT_LowStock`, which opens or closes LOW_STOCK_ALERT rows. The app does neither.

Writes in one transaction. Price edit: PRODUCT `Sell_price` plus a PRICE_CHANGE row. Supplier order: SUPPLIER_ORDER, SUPPLIER_ORDER_ITEM, then `LOW_STOCK_ALERT.Order_id`.

Not drawn. The EMPLOYEE self reference (`Supervisor_id`), PRODUCT_RETURN (the dashboard counts the 'requested' ones, read only), and other parts' tables that point at PRODUCT, DISCOUNT, CATEGORY or EMPLOYEE (for example TRANSACTION_ITEM, CART_ITEM, REVIEW). Those links are why a product or discount with history cannot be hard deleted.